Latency-Constrained Sequence Labeling ON CPU

Nikita Tiurkov **Innopolis University**

Motivation

Modern NLP relies on massive, rigorously preprocessed datasets, which often include **sequence labeling** as an intermediate step, but these data-cleaning steps are typically relegated to CPU clusters due to the prohibitive cost and scarcity of GPUs. To maximize processing throughput under strict CPU constraints, this study benchmarks the speed, memory, and quality trade-offs of traditional statistical models (**HMM** and **CRF**), neural models (**BiLSTM-CRF** and **RoBERTa**) on two popular sequence labeling tasks (POS Tagging and NER). Furthermore, I evaluate the impact of vital inference optimizations, including standard and **smart batching**, dynamic and weight-only **quantization**, and **ONNX** Runtime acceleration. Ultimately, this work provides actionable model selection guidelines for achieving cost-effective, high-throughput sequence labeling in industry environments.

Methodology

To evaluate sequence labeling performance, I benchmarked traditional models (**HMM** and **CRF**) **trained from scratch** against **pre-trained** neural architectures (**BiLSTM-CRF** and **RoBERTa**). Although comparing pre-trained models to locally trained baselines is imperfect for absolute accuracy evaluations, it is acceptable here since my **primary focus is throughput**, using accuracy merely to illustrate the speed-versus-quality trade-off. To identify optimal configurations, I evaluated the neural models using both **unbatched** and **bulk processing** (batch size 128) strategies, whereas traditional models were strictly evaluated sequentially by design. Additionally, I implemented a **smart batching** technique for the neural pipelines, sorting input sentences by length prior to inference to **minimize padding tokens**. I further accelerated the neural architectures utilizing **8-bit dynamic and weight-only quantization** (torchao), PyTorch **graph compilation** (torch.compile), and **ONNX** Runtime graph optimizations. Ultimately, all configurations were constrained to 8 CPU threads and rigorously profiled to record **latency** percentiles (p50, p90, p99), sequence **throughput**, **peak memory usage**, and Macro/Micro **F1 scores**.

Experimental Setup

Datasets		Tasks	
UD-English-EWT	**OntoNotes5**	**POS Tagging**	**NER**

Models				Optimizations		
HMM	**CRF**	**BiLSTM-CRF**	**RoBERTa**	**Batching**	**Quantization**	**ONNX**

Results

Figure 1 reveals a task-dependent Pareto frontier: for POS tagging, the **CRF achieves both the highest macro F1** (0.913) **and peak throughput** (30,402 seq/s), making it Pareto-optimal with no trade-off to manage. For NER, however, no single model dominates – the **CRF leads in throughput** (15,203 seq/s) **but trails in quality** (macro F1: 0.690), while the **Transformer achieves the highest F1** (0.902) **at the cost of throughput**, presenting a strict speed-quality trade-off.

Among inference optimizations, **smart batching consistently delivers the largest gains**: on NER, it reduces the Transformer's p50 latency from 9.0ms to 4.0ms while simultaneously cutting peak memory from 1,151MB to 316MB, with no accuracy loss. **Weight-only quantization further reduces memory** to 257MB and marginally improves tail latency (p99: 75ms), preserving F1 within 0.001 of the unquantized baseline. Conversely, **dynamic quantization proves counterproductive**, inflating Transformer p50 latency to 47ms on NER – slower than unbatched inference – and introducing severe p99 variance (Figure 2), making it unsuitable for latency-sensitive deployments. Similarly, **ONNX Runtime fails to outperform the optimized PyTorch baseline** on any latency or throughput metric.

As shown in Figure 3, memory constitutes the stricter operational bottleneck: the Transformer under standard batch-128 consumes 1,151MB on NER, **posing the highest OOM risk**, whereas **smart batching and weight-only quantization reduce this** to a safe operational margin. **Classical models** (CRF, HMM) **remain negligible in memory footprint** (<2MB) across all configurations.

Figure 1. Pareto frontier of model quality versus throughput under CPU constraints. While the classical CRF model dominates POS tagging in both speed and accuracy, NER presents a strict trade-off where Transformers achieve the highest F1 scores at the expense of throughput. Note that all advanced optimization techniques (smart batching, quantization, and ONNX) were evaluated on top of a baseline batch size of 128 to ensure efficient CPU utilization.

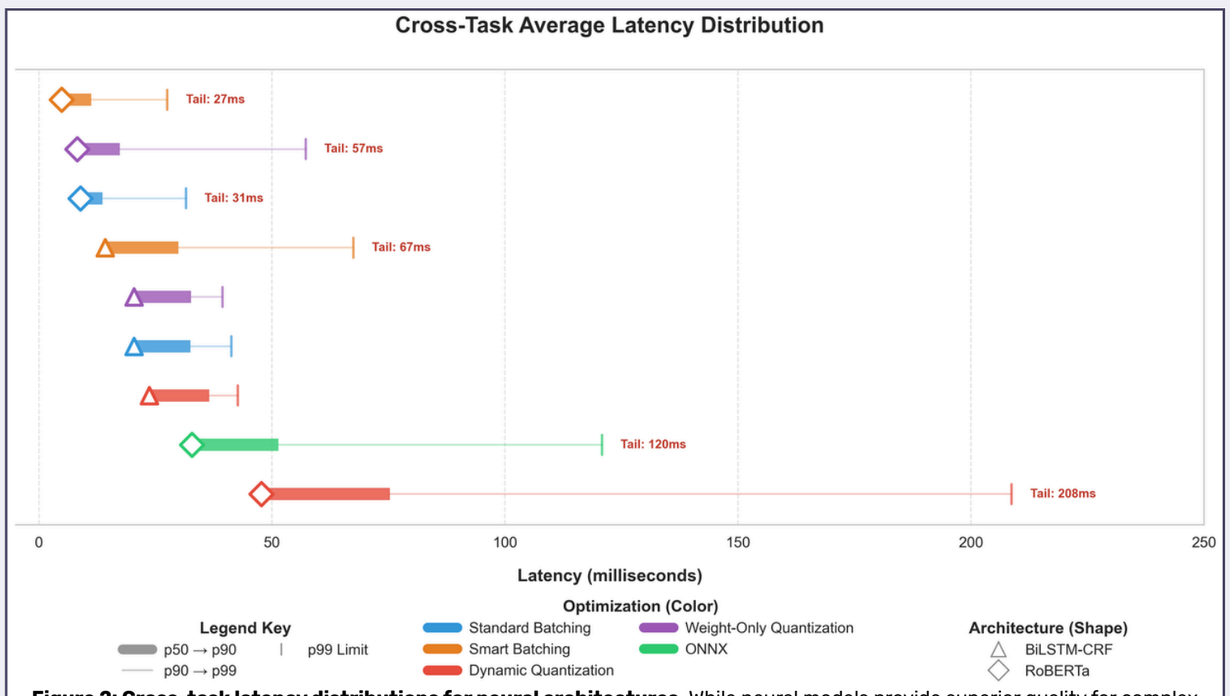

Figure 2: Cross-task latency distributions for neural architectures. While neural models provide superior quality for complex tasks, they introduce significant computational overhead. Latencies are averaged across both NER and POS tasks. For real-time scenarios with strict timing constraints, weight-only quantization and smart batching emerge as the recommended optimizations, as they provide the most stable median-to-tail performance and avoid the severe p99 latency spikes observed with dynamic quantization.

Conclusion

This study shows that model selection for CPU-constrained sequence labeling is **inherently task-dependent**. For POS tagging, the CRF is the clear leader, offering state-of-the-art quality and throughput at negligible memory cost. For NER, the optimal choice depends on operational constraints: the CRF remains the throughput champion, while the Transformer with smart batching is the only configuration achieving high F1 (0.902) under practical latency budgets. Among further optimizations, **only weight-only quantization is worth applying on CPU**. Ultimately, **smart batching should be the first optimization** applied to any neural pipeline, as it simultaneously improves latency, throughput, and memory footprint at zero accuracy cost.

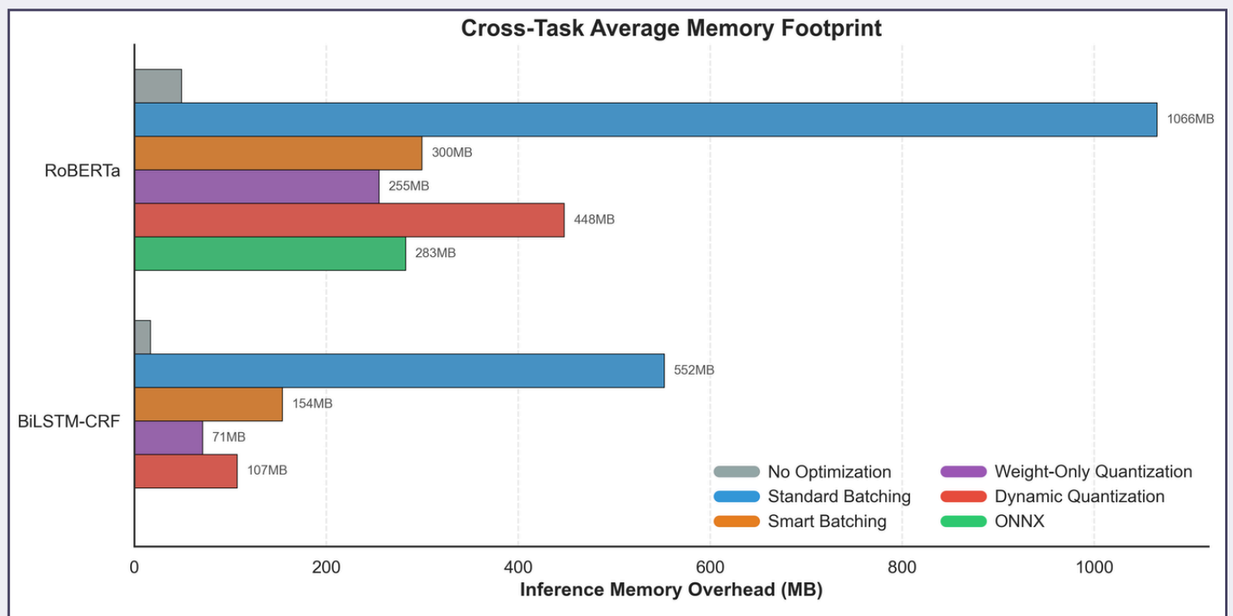

Figure 3: Cross-task memory footprint for neural architectures. Memory serves as a stricter operational bottleneck than latency; whereas high latency results in processing delays, exceeding memory capacity leads to critical Out-of-Memory failures. Memory utilizations were averaged during inference across the entire test set for both the POS tagging and NER tasks. RoBERTa with standard batching poses the highest risk of failure, whereas optimizations like smart batching and quantization significantly improve the safety margin for deployment.

Limitations

Several limitations warrant consideration when interpreting these results. First, conclusions are drawn from **only two tasks and two datasets**, and **may not generalise to other sequence labeling domains**. Second, the quality comparison between traditional and neural models is confounded by training data: **HMM and CRF are trained from scratch** on task-specific corpora, whereas **BiLSTM-CRF and RoBERTa leverage large-scale pretraining**, making absolute F1 differences partially attributable to data advantage rather than architecture alone. Third, all **experiments were conducted on an Apple M1 chip**; the poor performance of dynamic quantization and ONNX Runtime likely reflects **limited operator support on this platform**, and results may differ substantially on x86 hardware. Finally, scaling behaviour beyond 8 CPU threads remains untested.